

Scaling Laws of Graph Neural Networks for Atomistic Materials Modeling*

Chaojian Li⁴, Zhifan Ye⁴, Massimiliano Lupo Pasini¹, Jong Youl Choi², Cheng Wan⁴, and Prasanna Balaprakash³

¹Computational Sciences and Engineering Division, Oak Ridge National Laboratory

²Computer Science and Mathematics Division, Oak Ridge National Laboratory

³Computing and Computational Sciences Directorate, Oak Ridge National Laboratory

⁴Georgia Institute of Technology

{cli851, zye327, cwan39}@gatech.edu, {lupopasinim, choij, pbalapra}@ornl.gov

Abstract—Atomistic materials modeling is a critical task with wide-ranging applications, from drug discovery to materials science, where accurate predictions of the target material property can lead to significant advancements in scientific discovery. Graph Neural Networks (GNNs) represent the state-of-the-art approach for modeling atomistic material data thanks to their capacity to capture complex relational structures. While machine learning performance has historically improved with larger models and datasets, GNNs for atomistic materials modeling remain relatively small compared to large language models (LLMs), which leverage billions of parameters and terabyte-scale datasets to achieve remarkable performance in their respective domains. To address this gap, we explore the scaling limits of GNNs for atomistic materials modeling by developing a foundational model with billions of parameters, trained on extensive datasets in terabyte-scale. Our approach incorporates techniques from LLM libraries to efficiently manage large-scale data and models, enabling both effective training and deployment of these large-scale GNN models. This work addresses three fundamental questions in scaling GNNs: the potential for scaling GNN model architectures, the effect of dataset size on model accuracy, and the applicability of LLM-inspired techniques to GNN architectures. Specifically, the outcomes of this study include (1) insights into the scaling laws for GNNs, highlighting the relationship between model size, dataset volume, and accuracy, (2) a foundational GNN model optimized for atomistic materials modeling, and (3) a GNN codebase enhanced with advanced LLM-based training techniques. Our findings lay the groundwork for large-scale GNNs with billions of parameters and terabyte-scale datasets, establishing a scalable pathway for future advancements in atomistic materials modeling.

Index Terms—AI/ML Application and Infrastructure; AI/ML System and Platform Design.

I. INTRODUCTION

Atomistic material modeling is essential for the discovery of new materials with target properties because it accelerates the process by predicting material properties with sufficiently

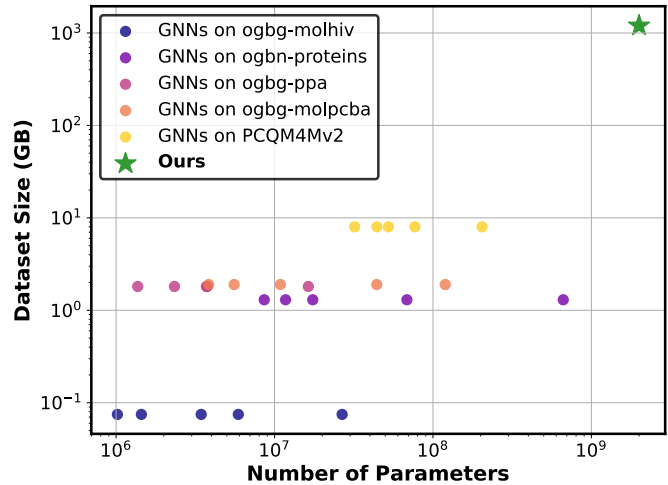


Fig. 1. Comparison of large-scale GNNs on multiple commonly-used biology/chemistry materials modeling datasets [12] with the foundational GNN developed in this work (indicated by a green star), after scaling both the model size and dataset size.

accurate results using atomistic information, without requiring costly full first-principles calculations [22], [24]. Neural networks play an important role in atomistic material modeling by training on data collected from chemistry experiments and first-principles calculations, formalizing material modeling as a neural network inference process [1]. Among these neural networks, Graph Neural Networks (GNNs) [16], [30], [36] have emerged as the state-of-the-art (SOTA) approach because atomistic material structures can be naturally mapped onto a graph, where atoms and interatomic bonds are treated as the nodes and edges of a graph, respectively [4], [32].

However, the great promise of GNNs for atomistic material modeling has not yet been fully realized because the potential of larger GNN models and more extensive graph datasets has not been fully exploited [19], [20], [32]. Specifically, as highlighted in recent studies on language and image-based neural networks [14], [42], larger neural networks trained on more comprehensive datasets tend to better capture the complex relationships in input data, leading to improved accuracy and reduced prediction errors. This scalability is often described by scaling laws, which indicate that increasing model size and

*This manuscript has been authored in part by UT-Battelle, LLC, under contract DE-AC05-00OR22725 with the US Department of Energy (DOE). The US government retains and the publisher, by accepting the article for publication, acknowledges that the US government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this manuscript, or allow others to do so, for US government purposes. DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (<http://energy.gov/downloads/doe-public-access-plan>).

dataset size generally improves prediction quality. In contrast, GNNs for atomistic material modeling currently lag behind in terms of the scale of models and datasets used compared to language and image-based neural networks. For example, as shown in Fig. 1, prior scaled-up GNNs are typically trained on datasets in the range of megabytes or gigabytes and have millions of parameters [20], [24], whereas commonly used language models today are trained on terabyte-scale datasets and contain billions of parameters [33].

The aforementioned gap raises a series of fundamental questions about scaling up GNNs: (1) What is the potential of scaling GNN model architectures when the number of parameters reaches the billion level? (2) What is the effect of dataset size on achieved accuracy or prediction errors when scaled to the terabyte level? (3) Can prior techniques for scaling up distributed training of large-scale language and image-based neural networks also be applied to GNNs?

To address the questions raised above, we progressively scale up GNN model sizes from millions of parameters to billions, alongside a corresponding increase in the atomistic materials modeling dataset size to the terabyte level, as depicted in Fig. 1. In summary, our contributions are threefold:

- **Insight:** Extracted scaling laws for GNNs in atomistic materials modeling, offering guidance on the relationship between GNN model sizes, the amount of available training data, and the prediction error of material properties.
- **Model:** A foundational GNN model with billion-level parameters and terabyte-scale data, which, to the best of our knowledge, represents the largest GNN developed for atomistic materials modeling to date.
- **Infrastructure:** A scalable GNN training codebase integrated with SOTA distributed training techniques from language and image-based neural networks, enabling efficient training of billion-level GNNs with terabyte-scale atomistic materials modeling data.

II. RELATED WORKS

A. GNN in Atomistic Materials Modeling

GNNs have emerged as a powerful tool for atomistic materials modeling, offering state-of-the-art (SOTA) performance across various tasks in chemistry and materials science [2], [29]. These models leverage the inherent graph-like atomistic structures, where atoms are represented as nodes and chemical bonds as edges, allowing for a natural and intuitive representation of atomistic materials [40]. GNNs excel in capturing complex relationships between atoms and their local chemical environments, enabling them to learn meaningful representations that can be used to predict a wide range of atomistic materials' properties [23]. Recent advancements in GNN architectures have led to significant improvements in prediction accuracy, with models like EGNN [30] achieving chemical accuracy on benchmark datasets such as QM9 [28]. These models have demonstrated their ability to predict quantum mechanical properties, solubility, toxicity, and other crucial molecular characteristics with high precision. The success of

GNNs in property prediction has opened up new possibilities for accelerating drug discovery, materials design, and other fields where understanding structure-property relationships is crucial. However, the challenge remains in scaling these models to handle even larger datasets and more complex atomistic structures, a gap this paper aims to address.

B. GNN Foundation Model

Graph Foundation Models (GFM) represent a significant advancement in graph representation learning, designed to learn transferable representations that can generalize across diverse graph structures and tasks [19]. These models aim to overcome the traditional limitations of task-specific GNNs by learning universal graph representations that can be applied to previously unseen graphs and tasks. Notable examples include GraphAny [44], which successfully demonstrates the ability to perform node classification across multiple graphs with varying feature dimensions and class numbers. In the realm of atomistic materials modeling, HydraGNN-GFM [24] has emerged as a pioneering GFM architecture, specifically designed for large-scale scientific applications. HydraGNN-GFM's unique features include multi-task learning capabilities, flexible message passing neural network layers, and scalable distributed training support, allowing it to process hundreds of millions of graphs efficiently across thousands of GPUs. This scalability has been demonstrated through near-linear strong scaling performance on major supercomputing systems, processing over 154 million atomistic structures while maintaining high prediction accuracy. Motivated by the scalability demonstrated by HydraGNN-GFM, we adopt the same GNN model architecture for our scaling experiments in this work.

C. Scalable Training Techniques

Scalable training techniques are crucial for enabling GNNs to handle large-scale graphs that exceed the memory capacity of a single machine. Traditional distributed training approaches, widely adopted in deep learning [37], primarily rely on data parallelism, which distributes data batches across multiple devices while maintaining full model replicas on each device, synchronizing gradients during updates. Optimizer-level parallelism methods such as ZeRO [25], further reduce memory requirements by distributing optimizer states across devices. Conversely, model parallelism partitions the model architecture itself across devices, necessitating careful management of intermediate activations. Another memory-saving strategy, activation checkpointing, lowers memory usage by rematerializing intermediate activations during backward propagation [6]. For GNNs, distributed training introduces unique challenges due to the interconnected nature of graph data. Partition-based methods, which divide the graph into sub-graphs distributed across devices, mitigate these challenges by distributing computational workloads but often incur significant communication overhead from exchanging neighbor features [43].

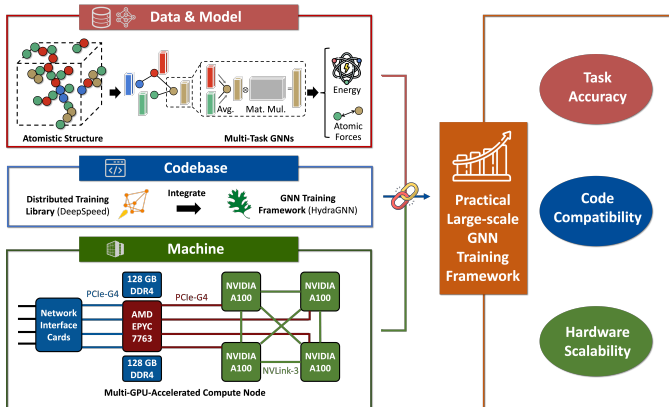


Fig. 2. An overview of the developed multi-stack infrastructure for scalable GNN training, integrating data and model configuration, refactored codebase, and multi-GPU machine setup. This unified framework aims to simultaneously improve GNN task accuracy through data-driven model design, ensure code modularity and reuse via codebase restructuring, and optimize training efficiency and scalability with a multi-GPU hardware architecture.

III. INFRASTRUCTURE SETUP

As summarized in Fig. 2, the entire infrastructure used for the scaling experiments includes not only the data and model but also the codebase supporting scalable GNN training and the high-performance computing (HPC) cluster.

A. Data

The dataset used in our scaling experiments is aggregated from multiple publicly available atomistic materials modeling datasets. As summarized in Tab. I, each dataset contains a different number of samples (graphs). Specifically, ANI1x [31] consists of up to 57 thousand distinct molecular configurations featuring the chemical elements C, H, N, and O. QM7-X [11] includes 42 physicochemical properties for approximately 4.2 million equilibrium and non-equilibrium structures of small organic molecules. OC2020-20M [4] is a subsampled set from the original OC2020 [4] dataset, spanning a range of oxide materials, coverages, and adsorbates. OC2022 [34] is an updated version of OC2020 [4], containing additional samples. MPTrj [13] focuses on atomistic structures of inorganic materials. Combining the five aforementioned data sources resulted in an aggregated dataset of 1.2 TB. We further sampled multiple datasets with sizes ranging from 0.1 TB to 1.2 TB from the original 1.2 TB dataset and used them in the scaling experiments. Following [20], [24], the corresponding tasks in the aggregated dataset are defined as predicting energy and atomic forces from atomistic structures represented in graph format. Specifically, the energy to be predicted is a property of the entire atomistic structure (graph), whereas the atomic forces are properties of individual atoms.

B. Model

To ensure equivalence with rotations, translations, reflections, and permutations in atomistic materials modeling, we

select EGNN [30] as the GNN model type, which is specifically designed for predicting molecular properties. To align with the task definitions in the aggregated dataset used for our scaling experiments, we add two types of output heads on top of the EGNN models: one for graph-level property prediction and the other for node-level property prediction. To study the effectiveness of the backbone model independently of task types when scaling data and model sizes, we vary only the depth and width of the EGNN backbone during the scaling experiments. The hyperparameter settings used for training follow those in [24], e.g., all models are trained for 10 epochs.

C. Codebase

We conduct the scaling experiments using HydraGNN [21], a commonly used GNN training framework for scientific discovery. However, as discussed in Sec. V, while HydraGNN supports data parallelism, directly scaling the model and data to sizes beyond those previously encountered in the codebase can exceed memory limits, even on high-performance HPC machines equipped with A100 GPUs. To address this limitation, we integrate DeepSpeed, a well-known distributed learning library, into HydraGNN to alleviate memory constraints, enabling us to scale our models to billions of parameters and handle terabyte-level datasets. The corresponding modifications have been committed to the official HydraGNN repository at: <https://github.com/ORNL/HydraGNN>.

D. Machine

We utilize Perlmutter [3], an HPC cluster consisting of A100-accelerated compute nodes. Each node is equipped with an AMD EPYC 7763 CPU, 256 GB of DDR4 memory, and four NVIDIA A100 GPUs interconnected via NVLink-3. Additionally, to accelerate the data loading process on the HPC machines, we employ the ADIOS [9] scientific data management library and DDStore [7], a distributed data store that facilitates in-memory data transfer between processes. In particular, we use 32 compute nodes for training each model.

IV. SCALING LAWS IN GNN

To examine the scaling laws for GNNs in atomistic material modeling, we conducted experiments on scaling up GNN model sizes (Sec. IV-A), increasing dataset size (Sec. IV-B), and analyzing how model depth and width affect the final test loss (Sec. IV-C). Specifically, model scaling is achieved by increasing the number of neurons in each layer, while dataset scaling is accomplished by adding samples to the training set

TABLE I
SUMMARY OF THE DATA SOURCES OF THE AGGREGATED DATASET USED IN THE SCALING EXPERIMENTS.

Data Source	# of Nodes	# of Edges	# of Graphs	Size
ANI1x [31]	75,700,481	1,050,357,960	4,956,005	25 GB
QM7-X [11]	70,675,659	1,020,408,506	4,195,237	25 GB
OC2020-20M [4]	1,538,055,547	33,734,466,610	20,994,999	726 GB
OC2022 [34]	705,379,388	18,937,505,384	8,834,760	395 GB
MPTrj [13]	49,286,440	729,940,098	1,580,227	17 GB

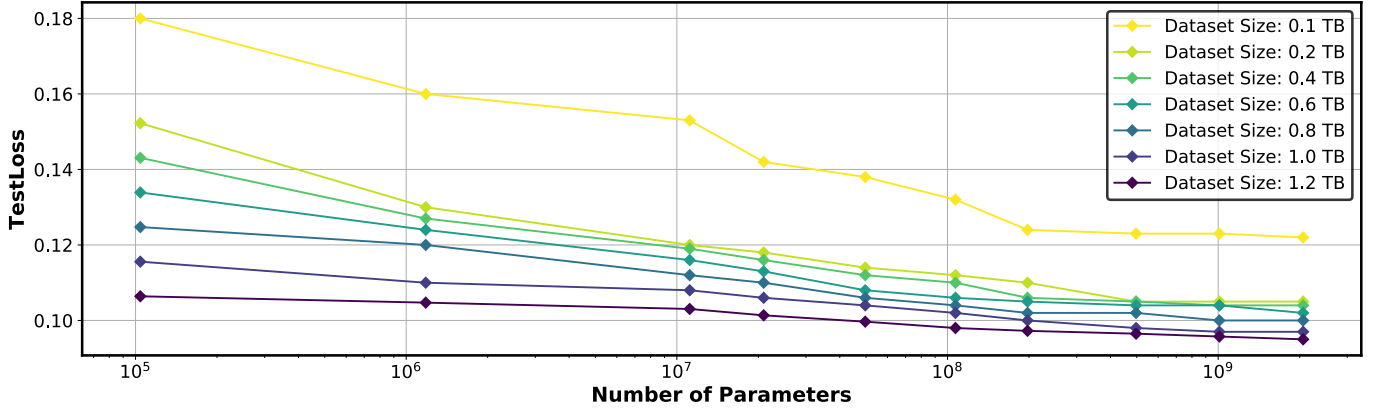


Fig. 3. The effect of scaling GNN **model** sizes across various dataset sizes on the final test loss.

from the aggregated 1.2 TB dataset. Additionally, the test loss for different model and dataset combinations is evaluated using the same held-out test set from the 1.2 TB dataset.

A. Model Scaling

As shown in Fig. 3, when the model size is scaled up by increasing the number of neurons in each layer, the test loss consistently decreases. This trend is observed across datasets of different sizes, ranging from 0.1 TB to 1.2 TB. However, unlike the observations in large-scale language and image-based models [14], [42], where the loss is nearly linear with the log-scale of the number of parameters, the decrease in test loss for GNNs in atomistic materials modeling shows diminishing returns as model size increases. We conjecture that this is due to architectural differences between Transformer models [35], commonly used in language and image-based tasks, and GNNs used in atomistic materials modeling. Specifically, Transformer models [35] rely on attention mechanisms, which can adaptively learn connections between different input samples (tokens). In contrast, GNN architectures, even advanced ones like EGNN [30] that account for rotational, translational, reflective, and permutational equivalences, are inherently limited by their locality constraints. These constraints restrict the ability to freely learn connections between

any pair of nodes, reducing their overall learning capacity compared to Transformers.

Although there has been some exploration of applying Transformers to graphs or encoding graphs directly into Transformer tokens [35], [41], their effectiveness in atomistic materials modeling remains underexplored. Furthermore, many of these approaches are limited to solving simple graph statistics-related problems [10], [39].

In conclusion, **further scaling of GNN model sizes is a promising direction for improving the quality of atomistic materials modeling.** However, when scaling beyond 2 billion parameters, the limitations of current GNN architectures may become a bottleneck.

B. Data Scaling

As summarized in Fig. 4, the test loss of GNNs with varying parameter counts, ranging from 0.1 million to 2 billion, consistently decreases as more data becomes available for training. In particular, when the dataset size increases from 0.1 TB to 0.2 TB, there is a noticeable drop in test loss compared to the more gradual decreasing trend observed beyond 0.2 TB. We conjecture that this is due to the significant differences between the 0.1 TB subset and the full 1.2 TB dataset. The test set is sampled from the complete 1.2 TB dataset and remains fixed across all experiments. As a result,

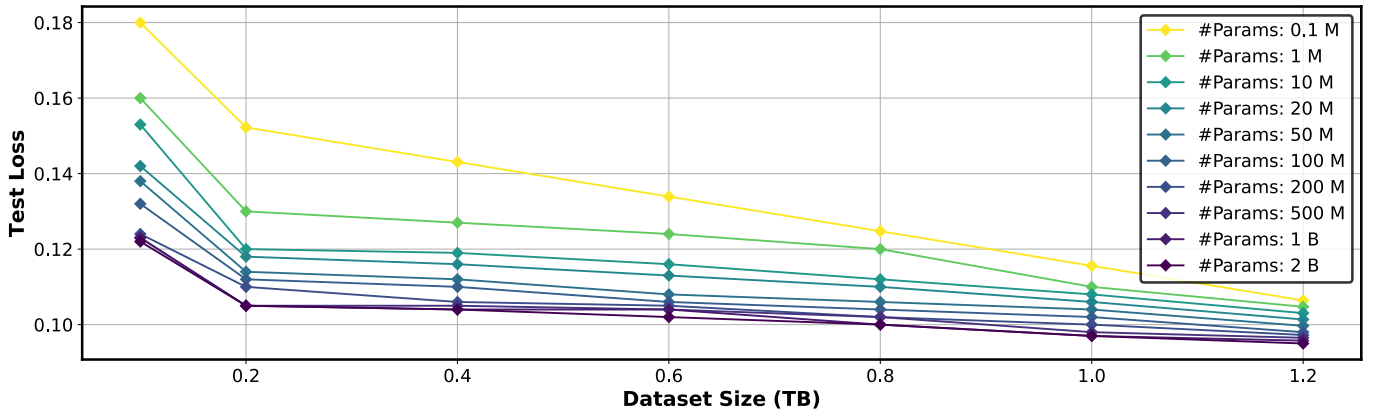


Fig. 4. The effect of scaling atomistic materials modeling **dataset** sizes across various GNN model sizes on the final test loss.

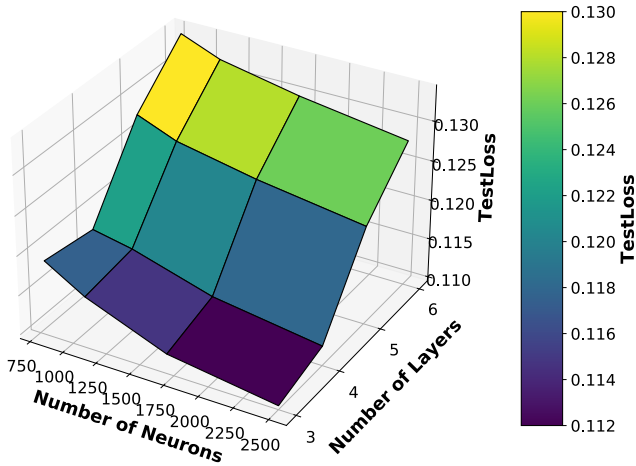


Fig. 5. Comparison of how scaling GNN model **depth** (i.e., number of layers) and **width** (i.e., number of neurons in each layer) affects the test loss when training on a dataset of 0.4 TB in size.

when only 0.1 TB is sampled for training, there is likely a mismatch between the distribution of the training dataset and the test set, leading to a relatively higher test loss for the 0.1 TB setting.

In contrast, as the dataset size gradually increases from 0.2 TB to 1.2 TB, the test loss decreases steadily and predictably, consistent with observations in existing large-scale language and image-based models [14], [42]. The more pronounced loss reduction, even at the 1.2 TB scale, suggests that **scaling data is more effective than scaling model size when both reach relatively large scales**, i.e., millions of parameters and terabytes of data.

However, it is important to note that scaling data is more challenging than scaling models. Simply increasing model depth and width produces a larger model, whereas scaling data involves tedious processes such as data collection, cleaning, and verification, as demonstrated in [24]. Therefore, the bottleneck in exploring the limits of scaling GNN models for atomistic materials modeling is similar to the bottleneck faced in scaling language and image-based models: the need for large-scale, high-quality data.

C. Model Depth vs. Width

As noted in previous works on scaling neural networks, a model’s depth and width are two critical factors that influence the achieved test loss [14], [42]. Examining the effect of depth and width in GNN models is particularly important, as it is widely recognized that building deep GNNs is more challenging compared to convolutional neural networks or Transformer models [17], [18]. Motivated by prior observations on scaling depth and width, we conducted experiments to explore how these two factors affect relatively large-scale GNN models with 10 to 100 million parameters on a substantial 0.4 TB dataset.

As summarized in Fig. 5, our findings indicate that increasing model width—i.e., the number of neurons in each

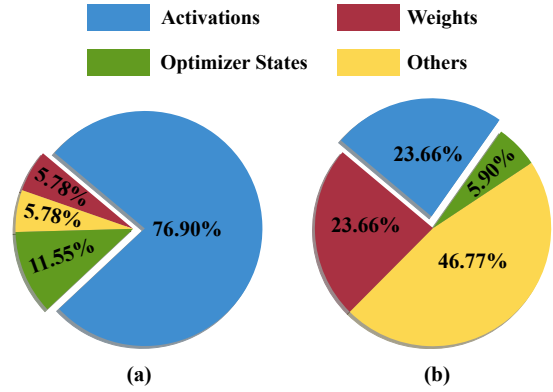


Fig. 6. Summary of the peak memory usage breakdown when training GNNs using (a) vanilla PyTorch-based HydraGNN [21] and (b) HydraGNN integrated with activation checkpointing and the ZeRO optimizer [25].

layer—consistently results in lower test loss. In contrast, increasing the number of layers beyond three leads to higher test loss, even when the total model size increases. We hypothesize that the over-smoothing [5] issue inherent to GNN architectures persists even at such large-scale dataset sizes and model capacities. This suggests that scaling GNN models by **increasing the number of neurons per layer is a more effective strategy than increasing the number of layers**.

V. EFFECTIVENESS OF TRAINING TECHNIQUES

To support training models with millions of parameters and datasets at terabyte scales, using the pure PyTorch-based data-distributed training in HydraGNN leads to out-of-memory issues, even on mainstream A100 GPUs. To address this, we investigated (1) the peak memory bottlenecks in large-scale GNN training (Sec. V-A) and (2) whether commonly used methods for scalability in large language models can be effectively applied to GNNs in atomistic material modeling (Sec. V-B and Sec. V-C).

A. Bottleneck Profiling

Limited memory capacity has long been a critical bottleneck in training large deep learning models [26]. To overcome this limitation, numerous memory-efficient training techniques have been proposed [6], [8], primarily targeting applications in image and text domains. However, the memory constraints in training large GNNs pose similar challenges, limiting their scalability [38]. To better understand and address these challenges, we conducted a memory bottleneck profiling analysis to evaluate the memory consumption in training foundational GNNs and to identify effective techniques for scalable GNN training. Since memory usage varies throughout the training process, our analysis focuses on profiling *peak memory usage*, defined as the highest memory consumption observed during training. This peak typically occurs across three stages: (1) the forward pass, (2) the backward pass, and (3) the weight updates performed by the optimizer.

TABLE II
REDUCTION IN PEAK MEMORY USAGE AND THE OVERHEAD OF TRAINING
TIME AFTER ADOPTING ACTIVATION CHECKPOINTING AND ZeRO
OPTIMIZER.

Setting	Relative Peak Memory	Relative Training Time
Vanilla Pytorch	100%	100%
+ Activation Checkpointing	42%	110%
+ ZeRO Optimizer	27%	133%

Fig. 6 (a) illustrates the peak memory usage breakdown during GNN training without any memory-efficient optimizations. The results indicate that the peak memory usage arises at the start of the backward pass. The breakdown highlights two key insights: (1) **Activations** dominate peak memory usage, accounting for 76.90% of the total. These tensors store intermediate values computed during the forward pass and are essential for gradient computation during the backward pass; (2) **Optimizer states** represent the second largest contributor to peak memory usage. These states, maintained by the Adam optimizer [15], include momentum vectors, which require storage equivalent to twice the size of the model weights.

B. Activation Checkpointing

To mitigate the substantial memory overhead caused by activation tensors, we implemented the activation checkpointing technique [6] in HydraGNN framework [21]. This method reduces memory consumption by selectively recomputing partial activations during the backward pass, rather than storing all intermediate activations generated during the forward pass. By applying this technique, activation tensors are no longer the dominant contributor to memory usage, resulting in a significant 58% reduction in peak memory usage. Following this optimization, the new peak memory usage shifts to the weight update phase. However, as shown in Tab. II, activation checkpointing comes at the cost of a 10% increase in training latency due to the overhead introduced by recomputation.

C. ZeRO Optimizer

After adopting activation checkpointing, optimizer states became the largest contributor to peak memory usage. To address this, we incorporated the ZeRO optimizer [25] by adding DeepSpeed library [27] in HydraGNN framework [21], which reduces memory demands by partitioning optimizer states across multiple GPUs instead of duplicating them on each device. With four GPUs within one compute node, this approach achieved a 36% reduction in peak memory usage compared to using activation checkpointing alone, as shown in Fig. 6. However, as elaborated in Tab. II, this optimization introduces additional cross-GPU communication overhead, leading to a 133% increased training runtime.

VI. IMPACT OF THE SCALED MODELS AND THE INFRASTRUCTURE

As demonstrated in Sec. IV-A, the scaled-up GNN models demonstrate improved accuracy over smaller versions. This improvement highlights the potential of scaling in addressing

the complexities of molecular property prediction tasks. The enhanced performance of these models could advance a range of scientific applications. For example, in material discovery, these models can enable rapid exploration of vast compositional and structural spaces, predicting key properties such as mechanical strength, thermal conductivity, and electronic behavior. By predicting critical properties, this capability accelerates the identification of novel materials for energy storage, catalysis, and sustainable manufacturing. Similarly, in drug design, the enhanced predictive power of these models can facilitate the identification of drug candidates with desired pharmacological properties, streamlining the traditionally time-consuming and resource-intensive process of lead optimization. Beyond these applications, the techniques and practices of the delivered infrastructure lay a foundation for scaling GNN models further and adapting them to broader applications in molecular science and other domains including biology, sociology, and more.

VII. CONCLUSION

This work bridges the gap between current GNNs for atomistic materials modeling and advances in scalable training techniques and scaling laws observed in large language and image-based models. By scaling GNNs to billions of parameters and terabyte-level datasets, we achieve significant improvements in predicting material properties. Our findings uncover scaling laws for GNNs, emphasizing the relationship between model size, dataset volume, and prediction accuracy, and establish a new benchmark in this field. The extracted insights, large-scale foundation models, and infrastructure developed in this work enable efficient handling of large-scale models and data, advancing atomistic materials modeling and expanding scientific applications.

ACKNOWLEDGEMENTS

Massimiliano Lupo Pasini would like to thank Dr. Vladimir Protopopescu for his valuable feedback in the preparation of the manuscript. This research is sponsored by the Artificial Intelligence Initiative as part of the Laboratory Directed Research and Development (LDRD) Program of Oak Ridge National Laboratory, managed by UT-Battelle, LLC, for the US Department of Energy under contract DE-AC05-00OR22725. This work used resources of the Oak Ridge Leadership Computing Facility, which is supported by the Office of Science of the U.S. Department of Energy, under INCITE award CPH161. This work also used resources of the National Energy Research Scientific Computing Center, which is supported by the Office of Science of the U.S. Department of Energy under Contract No. DE-AC02-05CH11231, under awards ERCAP0027259 and ERCAP0030519. This work was supported in part by the National Science Foundation (NSF) (Award ID: 2400511 and 2016727), the Department of Health and Human Services Advanced Research Projects Agency for Health (ARPA-H) under Award Number AY1AX000003, and CoCoSys, one of the seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA.

REFERENCES

- [1] R. M. Balabin and E. I. Lomakina, "Neural network approach to quantum-chemistry data: Accurate prediction of density functional theory energies," *The journal of chemical physics*, vol. 131, no. 7, 2009.
- [2] D. Buterez, J. P. Janet, S. J. Kiddle, D. Oglic, and P. Lió, "Transfer learning with graph neural networks for improved molecular property prediction in the multi-fidelity setting," *Nature Communications*, vol. 15, no. 1, p. 1517, 2024.
- [3] N. E. R. S. C. Center, "Perlmutter supercomputer," 2022. [Online]. Available: <https://www.nersc.gov/systems/perlmutter/>
- [4] L. Chanussot, A. Das, S. Goyal, T. Lavril, M. Shuaibi, M. Riviere, K. Tran, J. Heras-Domingo, C. Ho, W. Hu *et al.*, "Open catalyst 2020 (oc20) dataset and community challenges," *Acs Catalysis*, vol. 11, no. 10, pp. 6059–6072, 2021.
- [5] D. Chen, Y. Lin, W. Li, P. Li, J. Zhou, and X. Sun, "Measuring and relieving the over-smoothing problem for graph neural networks from the topological view," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 04, 2020, pp. 3438–3445.
- [6] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," *arXiv:1604.06174*, 2016.
- [7] J. Y. Choi, M. Lupo Pasini, P. Zhang, K. Mehta, F. Liu, J. Bae, and K. Ibrahim, "Ddstore: Distributed data store for scalable training of graph neural networks on large atomistic modeling datasets," in *Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*, 2023, pp. 941–950.
- [8] J. Duan, S. Zhang, Z. Wang, L. Jiang, W. Qu, Q. Hu, G. Wang, Q. Weng, H. Yan, X. Zhang, X. Qiu, D. Lin, Y. Wen, X. Jin, T. Zhang, and P. Sun, "Efficient training of large language models on distributed infrastructures: A survey," 2024.
- [9] W. F. Godoy, N. Podhorski, R. Wang, C. Atkins, G. Eisenhauer, J. Gu, P. Davis, J. Choi, K. Germaschewski, K. Huck *et al.*, "Adios 2: The adaptable input output system. a framework for high-performance data management," *SoftwareX*, vol. 12, p. 100561, 2020.
- [10] J. Guo, L. Du, H. Liu, M. Zhou, X. He, and S. Han, "Gpt4graph: Can large language models understand graph structured data? an empirical evaluation and benchmarking," *arXiv:2305.15066*, 2023.
- [11] J. Hoja, L. Medrano Sandomas, B. G. Ernst, A. Vazquez-Mayagoitia, R. A. DiStasio Jr, and A. Tkatchenko, "Qm7-x, a comprehensive dataset of quantum-mechanical properties spanning the chemical space of small organic molecules," *Scientific data*, vol. 8, no. 1, p. 43, 2021.
- [12] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open graph benchmark: Datasets for machine learning on graphs," *Advances in neural information processing systems*, vol. 33, pp. 22 118–22 133, 2020.
- [13] A. Jain, S. P. Ong, G. Hautier, W. Chen, W. D. Richards, S. Dacek, S. Cholia, D. Gunter, D. Skinner, G. Ceder *et al.*, "Commentary: The materials project: A materials genome approach to accelerating materials innovation," *APL materials*, vol. 1, no. 1, 2013.
- [14] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv:2001.08361*, 2020.
- [15] D. P. Kingma, "Adam: A method for stochastic optimization," *arXiv:1412.6980*, 2014.
- [16] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv:1609.02907*, 2016.
- [17] G. Li, M. Muller, A. Thabet, and B. Ghanem, "Deepgcns: Can gcns go as deep as cnns?" in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 9267–9276.
- [18] G. Li, C. Xiong, A. Thabet, and B. Ghanem, "Deepgcn: All you need to train deeper gcns," *arXiv:2006.07739*, 2020.
- [19] J. Liu, C. Yang, Z. Lu, J. Chen, Y. Li, M. Zhang, T. Bai, Y. Fang, L. Sun, P. S. Yu *et al.*, "Towards graph foundation models: A survey and beyond," *arXiv:2310.11829*, 2023.
- [20] J. Liu, H. Mao, Z. Chen, T. Zhao, N. Shah, and J. Tang, "Neural scaling laws on graphs," *arXiv:2402.02054*, 2024.
- [21] M. Lupo Pasini, S. T. Reeve, P. Zhang, J. Y. Choi, M. Lupo Pasini, S. T. Reeve, P. Zhang, and J. Y. Choi, "Hydragnn," oct 2021.
- [22] M. Lupo Pasini, G. Samolyuk, M. Eisenbach, J. Y. Choi, J. Yin, and Y. Yang, "First-principles data for solid solution niobium-tantalum-vanadium alloys with body-centered-cubic structures," *Scientific Data*, vol. 11, no. 1, p. 907, 2024.
- [23] H. Ma, Y. Bian, Y. Rong, W. Huang, T. Xu, W. Xie, G. Ye, and J. Huang, "Cross-dependent graph neural networks for molecular property prediction," *Bioinformatics*, vol. 38, no. 7, pp. 2003–2009, 2022.
- [24] M. L. Pasini, J. Y. Choi, K. Mehta, P. Zhang, D. Rogers, J. Bae, K. Z. Ibrahim, A. M. Aji, K. W. Schulz, J. Polo *et al.*, "Scalable training of graph foundation models for atomistic materials modeling: A case study with hydragnn," *arXiv:2406.12909*, 2024.
- [25] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," in *SC20*. IEEE.
- [26] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning," 2021.
- [27] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 3505–3506.
- [28] P. Reiser, M. Neubert, A. Eberhard, L. Torresi, C. Zhou, C. Shao, H. Metni, C. van Hoesel, H. Schopmans, T. Sommer *et al.*, "Graph neural networks for materials science and chemistry," *Communications Materials*, vol. 3, no. 1, p. 93, 2022.
- [29] J. G. Rittig, Q. Gao, M. Dahmen, A. Mitsos, and A. M. Schweidtmann, "Graph neural networks for the prediction of molecular structure-property relationships," 2023.
- [30] V. G. Satorras, E. Hoogeboom, and M. Welling, "E (n) equivariant graph neural networks," in *International conference on machine learning*. PMLR, 2021, pp. 9323–9332.
- [31] J. S. Smith, R. Zubatyuk, B. Nebgen, N. Lubbers, K. Barros, A. E. Roitberg, O. Isayev, and S. Tretiak, "The ani-1ccx and ani-1x data sets, coupled-cluster and density functional theory properties for molecules," *Scientific data*, vol. 7, no. 1, p. 134, 2020.
- [32] M. Sypetkowski, F. Wenkel, F. Poursafaei, N. Dickson, K. Suri, P. Fradkin, and D. Beaini, "On the scalability of gnns for molecular graphs," *arXiv:2404.11568*, 2024.
- [33] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv:2302.13971*, 2023.
- [34] R. Tran, J. Lan, M. Shuaibi, B. M. Wood, S. Goyal, A. Das, J. Heras-Domingo, A. Kolluru, A. Rizvi, N. Shoghi *et al.*, "The open catalyst 2022 (oc22) dataset and challenges for oxide electrocatalysts," *ACS Catalysis*, vol. 13, no. 5, pp. 3066–3084, 2023.
- [35] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.
- [36] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv:1710.10903*, 2017.
- [37] J. Verbraken, M. Wolting, J. Katzy, J. Kloppenburg, T. Verbelen, and J. S. Rellermeyer, "A survey on distributed machine learning," *Acm computing surveys (csur)*, vol. 53, no. 2, pp. 1–33, 2020.
- [38] C. Wan, Y. Li, A. Li, N. S. Kim, and Y. Lin, "Bns-gcn: Efficient full-graph training of graph convolutional networks with partition-parallelism and random boundary node sampling," *Proceedings of Machine Learning and Systems*, vol. 4, pp. 673–693, 2022.
- [39] L. Wei, Z. He, H. Zhao, and Q. Yao, "Unleashing the power of graph learning through llm-based autonomous agents," *arXiv:2309.04565*, 2023.
- [40] Z. Wu, J. Wang, H. Du, D. Jiang, Y. Kang, D. Li, P. Pan, Y. Deng, D. Cao, C.-Y. Hsieh *et al.*, "Chemistry-intuitive explanation of graph neural networks for molecular property prediction with substructure masking," *Nature Communications*, vol. 14, no. 1, p. 2585, 2023.
- [41] S. Yun, M. Jeong, R. Kim, J. Kang, and H. J. Kim, "Graph transformer networks," *Advances in neural information processing systems*.
- [42] X. Zhai, A. Kolesnikov, N. Houlsby, and L. Beyer, "Scaling vision transformers," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 12 104–12 113.
- [43] S. Zhang, A. Sohrabizadeh, C. Wan, Z. Huang, Z. Hu, Y. Wang, J. Cong, Y. Sun *et al.*, "A survey on graph neural network acceleration: Algorithms, systems, and customized hardware," *arXiv:2306.14052*, 2023.
- [44] J. Zhao, H. Mostafa, M. Galkin, M. Bronstein, Z. Zhu, and J. Tang, "Graphany: A foundation model for node classification on any graph," *arXiv:2405.20445*, 2024.